

4교시: 표준 실습 앱 이미지 만들기

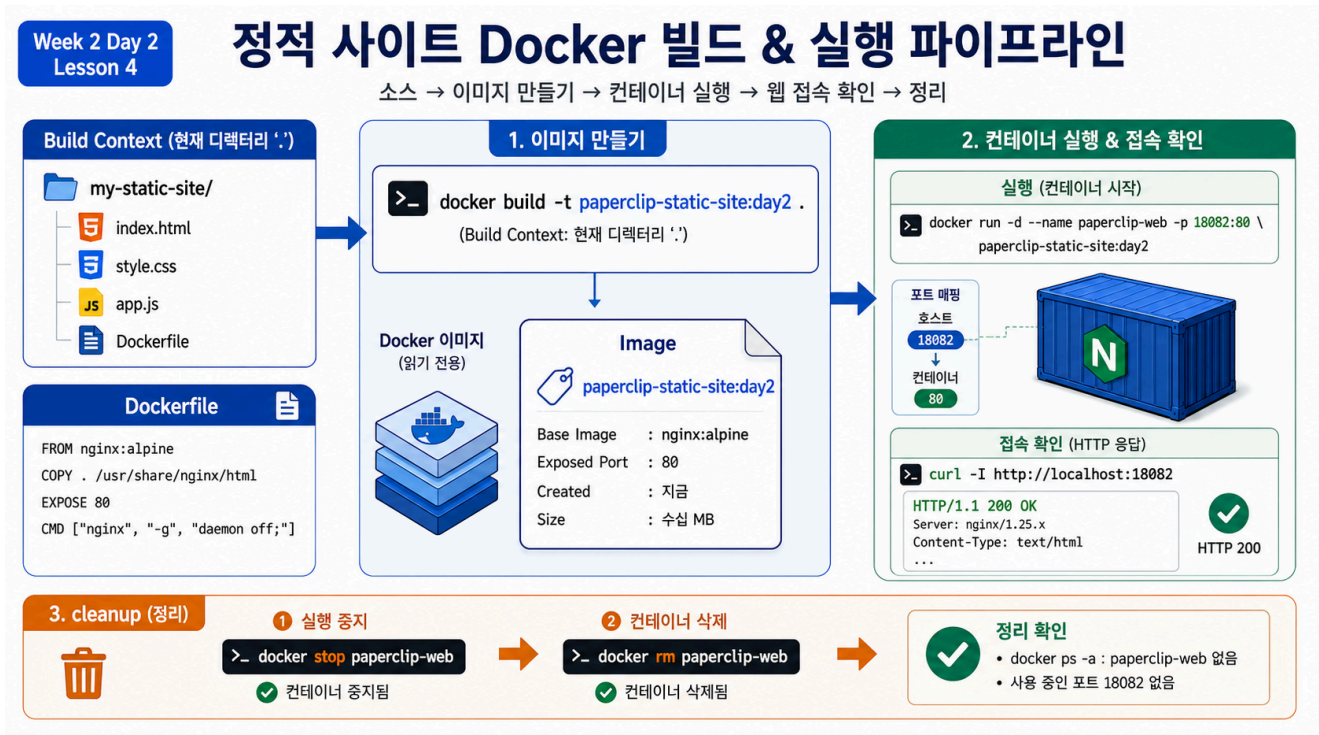
수업 목표

- docker build로 표준 실습 앱 image를 만든다.
- build output에서 context, Dockerfile step, image tag를 확인한다.
- build한 image를 container로 실행하고 HTTP 응답을 확인한다.

50분 흐름

시간	활동	비중	학생 산출
0-5분	build 명령과 tag 기준 확인	설명 10%	tag note
5-22분	docker build 실행	실행 35%	build output
22-35분	docker run과 docker ps 확인	실행 25%	container evidence
35-43분	curl HTTP 확인	실행 15%	HTTP evidence
43-50분	cleanup과 기록	실행 15%	cleanup note

Visual 1: Docker build/run pipeline



이 이미지는 source folder와 Dockerfile이 build context로 들어가 image tag를 만들고, 그 image에서 container를 실행해 HTTP 200으로 검증하는 흐름을 보여준다.

Hands-on

```
cd week2/day2/labs/static-site
docker build -t paperclip-static-site:day2 .
docker run -d --name paperclip-day2-static -p 18082:80 paperclip-static-site:day2
docker ps --filter name=paperclip-day2-static
curl -I http://localhost:18082
```

더 긴 실행 절차는 hands-on-lab.md의 Phase C와 Phase D를 따른다. 이 교시에서 시간이 남는 학생은 cache 재빌드까지 바로 진행한다.

Linux 사전 테스트 결과

Build 핵심 출력:

```
#4 [internal] load build context
#4 transferring context: 2.42kB
#6 [2/3] WORKDIR /usr/share/nginx/html
#7 [3/3] COPY index.html styles.css ./
#8 naming to docker.io/library/paperclip-static-site:day2 done
```

Run/HTTP 핵심 출력:

```
PORTS
0.0.0.0:18082->80/tcp, [::]:18082->80/tcp

HTTP/1.1 200 OK
Server: nginx/1.27.5
Content-Length: 1188
```

output 읽는 법

출력	의미	운영 판단
load build definition from Dockerfile	Dockerfile을 읽었다	build 시작 위치가 맞는가
load .dockerignore	제외 규칙을 읽었다	context 보호가 적용되는가
transferring context: 2.42kB	daemon에 전달된 파일 크기	불필요한 파일이 들어가지 않았는가
COPY index.html styles.css ./	source file을 image에 넣었다	앱 소스가 image 안에 포함되었는가
naming to ... paperclip-static-site:day2	tag가 붙었다	이후 run에서 사용할 이름이 명확한가

image build와 bind mount의 차이

Day 1과 Day 2 7교시의 bind mount는 host file을 container에 연결해 개발 중 변경을 빠르게 확인하는 방식이다. 반면 4교시의 Dockerfile build는 file을 image에 복사한다. 배포 관점에서는 image가 source file을 포함해야 같은 artifact를 다른 환경에서 실행할 수 있다. 개발 편의와 배포 재현성을 같은 것으로 혼동하지 않는다.

추가 실습: source 수정 후 rebuild

index.html의 hero 문장을 한 줄 바꾸고 다시 build/run한다.

```
docker build -t paperclip-static-site:day2-edit .
docker rm -f paperclip-day2-static
docker run -d --name paperclip-day2-static -p 18082:80 paperclip-static-site:day2-edit
curl -s http://localhost:18082 | grep "Dockerfile로 만든 표준 실습 앱"
```

확인할 것:

- source를 수정한 뒤에는 image를 다시 build해야 image 실행 결과가 바뀐다.
- bind mount처럼 host file 수정이 즉시 반영되는 구조가 아니다.
- day2와 day2-edit tag를 혼동하지 않도록 README에 tag를 정확히 기록한다.

핵심 유의사항

4교시는 처음으로 "내가 만든 image"를 실행하는 시간이다. 여기서 build 성공과 service 성공을 분리해야 한다. docker build가 성공했다는 것은 image artifact가 만들어졌다는 뜻이고, HTTP 200이 나온다는 것은 그 artifact로 실행한 container가 원하는 서비스를 제공한다는 뜻이다.

`docker run -d`는 background 실행이다. 명령이 바로 끝난다고 container가 종료된 것은 아니다. 그래서 바로 `docker ps`를 확인한다. 반대로 container ID가 출력됐는데 `docker ps`에 없으면 process가 곧바로 죽었을 수 있으므로 `docker ps -a`와 `docker logs`가 필요하다.

port 표기는 초급자가 가장 자주 헷갈린다. `-p 18082:80`에서 왼쪽 18082는 host port이고 오른쪽 80은 container 내부 port다. 브라우저나 `curl`은 host port인 18082로 접속한다. container 내부 nginx는 80번을 듣고 있다.

실습 중 자주 놓치는 것

놓치는 지점	증상	바로 확인할 명령
tag를 잘못 입력	Unable to find image locally	<code>docker images paperclip-static-site</code>
container name 중복	name already in use	<code>docker ps -a --filter name=paperclip-day2-static</code>
port 충돌	bind: address already in use	<code>docker ps</code> , 다른 host port 사용
build 후 run을 예전 tag로 실행	수정 내용이 안 보임	run 명령의 image tag 확인
<code>curl</code> 을 container port로 착각	localhost:80 접속 실패	<code>docker ps</code> 의 PORTS 확인

build/run/check를 한 세트로 묶기

build만 성공하고 넘어가지 않는다. 아래 세트를 한 번의 검증 단위로 본다.

```
docker build -t paperclip-static-site:day2 .
docker run -d --name paperclip-day2-static -p 18082:80 paperclip-static-site:day2
docker ps --filter name=paperclip-day2-static
curl -I http://localhost:18082
curl -s http://localhost:18082 | grep "Dockerfile로 만든 표준 실습 앱"
```

정상 기준:

- build 마지막에 `naming to docker.io/library/paperclip-static-site:day2 done`이 보인다.
- `docker ps`의 PORTS에 `0.0.0.0:18082->80/tcp`가 보인다.
- `curl -I`에서 `HTTP/1.1 200 OK`가 보인다.
- body check에서 실습 앱 제목이 보인다.

학생 기록 템플릿

```
## Lesson 4 Build Run Evidence
```

- build command:
- image tag:
- build context size:
- COPY step:
- run command:
- container name:
- host port:
- container port:
- HTTP status:
- expected body text:
- cleanup command:

빠른 학생 확장

시간이 남으면 host port만 바꾸어 같은 image를 다시 실행해 본다.

```
docker rm -f paperclip-day2-static
docker run -d --name paperclip-day2-static -p 18083:80 paperclip-static-
site:day2
curl -I http://localhost:18083
```

이 확장은 image는 그대로 두고 실행 조건만 바꿀 수 있다는 점을 보여준다. 같은 image라도 container name, host port, environment, volume 조건에 따라 실행 instance는 달라진다.

cleanup

```
docker stop paperclip-day2-static
docker rm paperclip-day2-static
```

평가 기준

기준	2점 evidence
build	image tag와 build output 핵심 step을 기록했다.
run	container name과 port binding을 기록했다.
HTTP	HTTP/1.1 200 OK를 확인했다.
cleanup	stop/rm으로 실습 container를 정리했다.

공식 근거 링크

- Docker build: <https://docs.docker.com/reference/cli/docker/buildx/build/>
- Docker run: <https://docs.docker.com/reference/cli/docker/container/run/>
- Dockerfile reference: <https://docs.docker.com/reference/dockerfile/>